Singleton Exercises

https://github.com/akarshiev

Purpose: Complete these exercises independently, add automated tests, and document how to run each solution. Starter code is intentionally omitted so the repository reflects your own implementation.

Exercise 1: Configuration Manager with Lazy Initialization

Implement a Java ConfigurationManager that loads application settings from config.properties. Only one instance should exist, created when first requested.

Requirements

- Use lazy Singleton initialization.
- Load settings using java.util.Properties.
- Provide getProperty(String key) and reloadConfig().
- Handle a missing file gracefully and document fallback behavior.
- Ensure reloadConfig refreshes the current settings.

Test and verify

- Repeated accessor calls return the same instance.
- Configured keys return expected values; missing keys have defined behavior.
- Missing-file behavior does not crash unexpectedly.

Exercise 2: Thread-Safe File Logger

Create a Singleton Logger that appends log entries to app.log, including when multiple threads write at once.

Requirements

- Use double-checked locking with safe publication of the instance.
- Implement log(String level, String message).
- Append entries in the format [TIMESTAMP] [LEVEL] message.
- Handle file I/O errors clearly.
- Prevent concurrent writes from interleaving or corrupting log entries.

Test and verify

- Concurrent access does not create multiple Logger instances.
- Existing log contents are preserved.
- A multithreaded test produces complete, timestamped lines.

Exercise 3: Enum-Based Cache Manager

Implement a CacheManager enum Singleton backed by ConcurrentHashMap, storing values with a time-to-live (TTL).

Requirements

- Use an enum Singleton such as CacheManager.INSTANCE.
- Implement put(String key, Object value, int ttlSeconds), get(String key), and remove(String key).
- Track expiration for each entry.

- Expired entries must not be returned, even before background cleanup runs.
- Use a background task to clean expired entries and keep operations thread-safe.
- Define behavior for zero or negative TTL values.

Test and verify

- Test storing, retrieving, and removing values.
- Verify expired values are treated as absent.
- Test concurrent reads and writes.

Exercise 4: Singleton Database Connection Pool

Create a Singleton DatabaseConnectionPool that manages multiple simulated database connections. The pool manager is one instance; connections are separate objects.

Requirements

- Limit the pool to 10 connections.
- Implement getConnection(), releaseConnection(Connection conn), and getPoolSize().
- Track available and borrowed connections.
- Synchronize access to avoid exceeding the limit or lending one connection twice.
- Apply a 30-second wait timeout when no connection is available.
- Handle invalid or foreign connections safely.

Test and verify

- The pool never exceeds 10 connections.
- Released connections can be reused.
- A waiting caller receives a clear timeout outcome after 30 seconds.

Exercise 5: Break and Protect a Singleton

Implement a SecurityManager Singleton that tracks login attempts, then investigate ways to create or simulate additional instances.

Requirements

- Implement a private constructor and accessor, plus login() and getLoginAttempts().
- Try breaking the implementation with Reflection.
- Test serialization/deserialization and implement readResolve().
- Test cloning and prevent cloning by overriding clone() appropriately.
- Investigate how separate class loaders can load separate copies of a class.
- Provide an enum-based implementation as the final robust alternative.

Test and verify

- Include reproducible tests and a short explanation for each attack.
- Verify the protections that can be tested.
- Explain why class-loader boundaries can permit separate class-level instances.

Bonus: Multithreaded Task Scheduler

Create a Singleton TaskScheduler shared by multiple threads.

Requirements

- Use thread-safe Singleton initialization.
- Use ScheduledExecutorService internally.
- Provide schedule(Runnable task, long delay, TimeUnit unit).
- Allow no more than five tasks to execute concurrently.
- Provide shutdown() and define behavior for submissions after shutdown.
- Document whether tasks run once or repeatedly and implement that contract.

Test and verify

- Concurrent calls return the same scheduler.
- The concurrency limit is enforced.
- Shutdown behavior is covered by tests.

Repository checklist

- Organize exercises into clear packages or modules and include a README with run instructions.
- Write unit tests for normal cases, edge cases, errors, and concurrency where relevant.
- Use meaningful names and document important design decisions.
- Do not commit logs, IDE files, credentials, or personal configuration.
- Compare lazy initialization, double-checked locking, and enum-based Singleton; discuss when Singleton may not be appropriate.